

Practice Questions on Neural Networks, CNNs, and Activation Functions

- Q1.** In a feedforward neural network, what ensures information flows only from input to output without loops?
- (a) Recurrent connections
 - (b) Weight sharing
 - (c) Directed acyclic structure
 - (d) Convolution
- Q2.** What is the main role of bias in a neuron?
- (a) To scale the input
 - (b) To shift the activation threshold
 - (c) To reduce overfitting
 - (d) To normalize input
- Q3.** The vanishing gradient problem is most severe when using:
- (a) ReLU
 - (b) Tanh
 - (c) Sigmoid
 - (d) Leaky ReLU
- Q4.** What is the primary benefit of parameter sharing in CNNs?
- (a) Faster backpropagation
 - (b) Reduced number of learnable parameters
 - (c) Increased receptive field
 - (d) Avoids overfitting completely
- Q5.** Which layer in a CNN usually helps in translation invariance?
- (a) Fully connected
 - (b) Convolution

- (c) Pooling
- (d) BatchNorm

Q6. If a CNN uses stride > 1 in convolution, the effect is:

- (a) Increased number of filters
- (b) Spatial downsampling
- (c) Reduced receptive field
- (d) More overlapping patches

Q7. Dilated convolutions are used primarily for:

- (a) Increasing receptive field without increasing parameters
- (b) Avoiding vanishing gradients
- (c) Reducing FLOPs
- (d) Increasing overfitting

Q8. Which activation function is most prone to the “dying neuron” problem?

- (a) ReLU
- (b) Leaky ReLU
- (c) ELU
- (d) Softmax

Q9. Why is Softmax often used in the output layer of classification networks?

- (a) It introduces non-linearity
- (b) It outputs probabilities summing to 1
- (c) It avoids vanishing gradients
- (d) It reduces dimensionality

Q10. Swish activation function is defined as:

- (a) $x \cdot \tanh(x)$

- (b) $x \cdot \sigma(x)$
- (c) $\max(0, x)$
- (d) $\sigma(x)(1 - \sigma(x))$

Q11. Batch Normalization primarily helps by:

- (a) Making gradients vanish slower
- (b) Reducing internal covariate shift
- (c) Increasing dataset size artificially
- (d) Eliminating need for activation functions

Q12. Dropout works by:

- (a) Randomly removing weights
- (b) Randomly setting some neuron outputs to zero
- (c) Reducing learning rate dynamically
- (d) Clipping gradient values

Q13. Which optimizer adapts learning rate for each parameter individually?

- (a) SGD
- (b) Momentum
- (c) Adam
- (d) NAG

Q14. Depthwise separable convolutions, as used in MobileNet, reduce:

- (a) FLOPs and parameters
- (b) Training dataset requirements
- (c) Receptive field size
- (d) Gradient vanishing

Q15. Residual connections in ResNet help primarily with:

- (a) Reducing training data need
- (b) Avoiding vanishing gradients

- (c) Increasing FLOPs
- (d) Enforcing weight sharing

Q16. Consider the following snippet:

```
import torch
import numpy as np
a=torch.randn(size=(2,2,2,3,4),dtype=torch.float64)
b=torch.randn(size=(2,2,3,4,3),dtype=torch.float64)
c=torch.matmul(a,b)
print(c.shape)
```

What will be the output of the program?

Q17. You instantiate the `SimpleCNN` model from the tutorial with `depth=4`. What is the value of `in_channels` for the last convolutional layer that is created inside the loop?

```
class SimpleCNN(nn.Module):
    def __init__(self, activation='relu', depth=2):
        super(SimpleCNN, self).__init__()
        # ... (rest of the init method) ...
        layers = []
        in_channels = 1
        for i in range(depth):
            out_channels = 7 * (3**i)
            layers.append(nn.Conv2d(in_channels, out_channels, ...))
            # ...
            in_channels = out_channels
```

Q18. You train a CNN on an image dataset with the following code:

```
# training
model.train()
for epoch in range(n):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
```

```

optimizer.zero_grad()

# evaluation
model.eval()
@torch.no_grad()
def get_accuracy(correct = 0, total = 0):
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = outputs.max(1)
        correct += (predicted == labels).sum().item()
    #... Rest of the code

```

Even for a large n , the model gives very low accuracy. Identify the reason and suggest changes.

- Q19.** You are given the CIFAR-10 dataset and a batch size of 128 images. What will be the output of the following snippet?

```

dataiter = iter(trainloader)
images, labels = dataiter.next()
print(images.shape)

```

- Q20.** Consider the following code snippet:

```

import torch
import torch.nn as nn

conv = nn.Conv3d(
    in_channels = 3, out_channels=64,
    kernel_size = (1, 3, 3),
    stride = (1, 1, 1),
    bias = False
)

```

Conceptually, along the temporal axis, this layer is equivalent to:

- (a) A 2D conv run on each frame separately (same weights for all frames)
- (b) A 1D conv over time
- (c) A 2D conv on each frame
- (d) A per-channel (depthwise) conv only, with no channel mixing